DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

1. What are the main differences between Python lists, tuples, and sets in terms of mutability, ordering, duplicates, and typical use cases?

Type	Mutable	Ordered	Allows Duplicates	Use Case
List	Yes	Yes	Yes	General-purpose collection where elements may change
Tuple	No	Yes	Yes	Fixed-size, immutable collections, often used for function returns or as keys in dictionaries
Set	Yes	No	No	Collection of unique items, optimized for fast membership testing

2. What is the difference between local and global variables in Python, and how does their scope affect program behaviour?

Local Variables:

- Defined **inside a function or block**.
- Accessible **only within that function**.
- Created when the function is called and destroyed when the function exits.

Global Variables:

Defined outside all functions, at the module level.

Accessible anywhere in the module (or even other modules if imported).

Persist throughout the program's lifetime.

Can be modified inside a function using the global keyword.

3. How do you detect and handle missing or null values in a dataset using Python (pandas)?

1. **Detect missing values:**

```
import pandas as pd

df = pd.DataFrame({

  "A": [1, 2, None, 4],
```

```
  "B": [None, 2, 3, 4]
})
# Check for missing values
df.isnull()
df.notnull()
```

2. **Fill missing values:**

```
# Replace missing values with a specific value (e.g., 0)
df.fillna(0, inplace=True)
```

3. **Drop missing values:**

```
# Drop rows that contain any null values
df.dropna(inplace=True)
# Drop columns with null values
df.dropna(axis=1, inplace=True)
```

4. What is the difference between a class method and a static method in Python, and when would you use each?

Class Method (@classmethod)

- Takes the **class itself** as the first argument (conventionally cls).
- Can access or modify **class-level attributes**.
- Useful for **factory methods** or methods that affect the class as a whole.

Static Method (@staticmethod)

- Does **not take self or cls** as a parameter.
- Cannot access instance or class attributes.
- Used for **utility functions** related to the class but independent of its state.

5. What is data serialization and deserialization in Python, and how can you perform it using JSON and Pickle?

- **Serialization:** Converting a Python object into a **byte stream** (or string) so it can be **stored or transmitted**.

- **Deserialization:** Converting the **byte stream back into a Python object**.

Using JSON (for standard data types like dict, list, str, int):

```
Import
 jsondata = {"name": "Alice", "age": 30}
# Serialize (Python object → JSON string)
json_string = json.dumps(data)
# Deserialize (JSON string → Python object)
restored_data = json.loads(json_string)
```

Using Pickle (for Python-specific objects, including non-JSON types):

```
import pickle
data = {"name": "Alice", "age": 30}
# Serialize and save to file
with open("data.pkl", "wb") as f:
   pickle.dump(data, f)
# Load and deserialize from file
with open("data.pkl", "rb") as f:
   loaded_data = pickle.load(f)
```

6. ***What is the difference between a shallow copy and a deep copy in Python, and when would you use each?***

Answer:

- **Shallow Copy:**
    - Creates a **new object**, but does **not create copies of nested objects**; references are shared.
    - Modifying a nested object in the copy **affects the original**.
    - Created using copy.copy() or slicing for lists.
- **Deep Copy:**
    - Creates a **new object and recursively copies all nested objects**.

- Modifications in the copy **do not affect the original**.
- Created using copy.deepcopy().

7. How can you read from and write data to a database in Python? Provide examples using SQLAlchemy and pandas.

In Python, you can interact with databases using libraries like **pyodbc**, **psycopg2**, or **SQLAlchemy**. A common approach is using **pandas** with **SQLAlchemy** for easy data handling.

Example using SQLAlchemy and pandas:

```
from sqlalchemy import create_engine

import pandas as pd

# Create a database connection (example: MSSQL)

engine = create_engine("mssql+pyodbc://username:password@DSN_NAME")

# Read data from a table into a pandas DataFrame

df = pd.read_sql("SELECT * FROM employees", engine)

# Write DataFrame back to a table (replace if it exists)

df.to_sql("employees_backup", engine, if_exists="replace", index=False)
```

8. What techniques can you use to optimize Python code for better performance?

To improve Python code performance, consider the following techniques:

1. **Use built-in functions and standard libraries:**
   - Functions like sum(), min(), max(), map(), and filter() are implemented in C and are faster than manual loops.
2. **Use generators for large datasets:**
   - Generators (yield) avoid creating large lists in memory.
3. squares = (x*x for x in range(1000000)) # generator expression
4. **Avoid global variables:**
   - Accessing global variables is slower; prefer local variables.
5. **Prefer list comprehensions and dictionary/set comprehensions:**

- They are faster than equivalent for loops.

6. squares = [x*x for x in range(10)]
7. **Use efficient data structures:**
    - Use dict or set for fast lookups instead of lists.
8. **Profile your code:**
    - Use cProfile, timeit, or line_profiler to identify performance bottlenecks.
9. import timeit

timeit.timeit("sum(range(1000))", number=1000)

9. What are Python list comprehensions, and how can you use them to create lists efficiently? Provide an example with a condition.

- **Definition:**
  List comprehensions provide a **concise and readable way** to create lists in Python. They combine a **for loop** and an **optional condition** in a single line.
- **Basic Example:**

Squares of numbers from 0 to 9

squares = [x**2 for x in range(10)]

print(squares) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

- **Example with Condition:**

Squares of even numbers only

even_squares = [x**2 for x in range(10) if x % 2 == 0]

print(even_squares) # [0, 4, 16, 36, 64

10. How do you handle exceptions in Python, and how can you raise custom exceptions? Provide examples.

- **Exception Handling:**
  Python uses **try, except, else, and finally** blocks to handle runtime errors and prevent program crashes.

try:

```
    x = 10 / 0  # This will raise ZeroDivisionError
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:  # Catch all other exceptions
    print("Error:", e)
else:
    print("No errors occurred.")
finally:
    print("This block always runs.")
```

- **Raising Exceptions:**
  You can manually raise exceptions using the raise keyword:

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

11. What are context managers in Python, and how are they used for resource management? Provide examples.

- **Definition:**
  Context managers in Python help **manage resources** by ensuring proper **setup and cleanup**, such as opening and closing files, acquiring and releasing locks, or connecting and disconnecting from databases. They are typically used with the with statement.
- **Example with files:**

```
# Using a context manager to open a file
with open("file.txt", "r") as file:
    content = file.read()
```

12. How can Python be integrated with big data tools like Apache Spark, and what are common use cases?

- **Integration with Apache Spark:**
  Python can work with big data tools using PySpark, the Python API for Apache Spark. It allows you to perform distributed data processing across clusters.
- Example:

```
from pyspark.sql import SparkSession

# Create a Spark session

spark = SparkSession.builder.appName("Example").getOrCreate()

# Read a CSV file into a Spark DataFrame

df = spark.read.csv("data.csv", header=True, inferSchema=True)

# Select columns and show data

df.select("name", "age").show()
```

- **Use Cases:**
  - ETL pipelines: Transform and load large datasets efficiently.
  - Machine Learning: Distributed ML algorithms using MLlib.
  - Big Data Analytics: Process massive datasets that do not fit into memory on a single machine

13. What is the role of the pandas library in Python for data manipulation, and what are its key features? Provide a short example.

Answer:

- **Definition:**
  Pandas is a **powerful Python library** for **structured data manipulation and analysis**, providing easy-to-use data structures and tools.
- **Key Features:**
  1. **DataFrames and Series:** Tabular and single-column data structures for efficient handling.
  2. **File operations:** Read/write CSV, Excel, SQL, JSON, and more.
  3. **Missing data handling:** Functions like fillna(), dropna().
  4. **Data grouping and aggregation:** groupby, pivot_table, merge, join.

5. **Time-series support:** Date/time indexing, resampling, and rolling windows.

- **Example:**

```
import pandas as pd

# Read CSV file

df = pd.read_csv("employees.csv")

# Calculate total compensation

df['total'] = df['salary'] + df['bonus']

# Compute average total per department

grouped = df.groupby('department')['total'].mean()

print(grouped)
```

14. **What are Python's key features?**

Answer:

- **Interpreted:** Python executes code line by line.
- **High-level:** Easy-to-read syntax.
- **Dynamically typed:** No need to declare variable types.
- **Object-oriented:** Supports classes, objects, inheritance, and polymorphism.
- **Extensive libraries:** Built-in and third-party libraries for data, web, ML, etc.
- **Cross-platform:** Runs on Windows, macOS, Linux

15. **Explain Python decorators**

Answer:

- Decorators are **functions that modify the behavior of other functions or classes**.
- Syntax: @decorator above the function.

```
def decorator(func):
  def wrapper():
```

```
        print("Before function")
        func()
        print("After function")
    return wrapper

@decorator
def say_hello():
    print("Hello")
say_hello()
```

16. **Explain Python's exception handling**

Answer:

- Use **try-except-finally** blocks to catch and handle errors.
- raise allows **manual exception throwing**.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Always executes")
```

17. **How to integrate Python with Apache Spark**

Answer:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Example").getOrCreate()
df = spark.read.csv("data.csv", header=True, inferSchema=True)
df.select("name", "age").show()
```

- **Use Case:** Distributed ETL, ML, analytics on big data.

18. **Explain Python's context managers**

Answer:

- Manage resources like files, DB connections using **with**.
- Ensures proper cleanup even if errors occur.

```
with open("file.txt", "r") as f:
  content = f.read()
# file auto-closed here
```

PYTHON CODING QUEATIONS:

1. Write a Python generator to produce an infinite Fibonacci sequence.

```
def fibonacci_generator():
  a, b = 0, 1
  while True:
    yield a
    a, b = b, a + b
# Example usage: print first 10 numbers
fib_gen = fibonacci_generator()
for _ in range(10):
```

```
  print(next(fib_gen))
```

2. Write a Python generator to produce prime numbers indefinitely.

```
def prime_generator():
  num = 2
  while True:
    for i in range(2, int(num ** 0.5) + 1):
      if num % i == 0:
        break
    else:
      yield num
    num += 1

# Example usage: print first 10 primes
primes = prime_generator()
for _ in range(10):
  print(next(primes))
```

3. Write Python code to sort a list without using the built-in sort() method.

```
# Using the sorted() function
numbers = [5, 2, 9, 1, 5, 6]
sorted_list = sorted(numbers)
print(sorted_list)

# Using Bubble Sort
numbers = [5, 2, 9, 1, 5, 6]
n = len(numbers)
for i in range(n):
```

```
  for j in range(0, n-i-1):
    if numbers[j] > numbers[j+1]:
      numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
print(numbers)
```

4. Write Python code to check whether a given string is a palindrome.

```
# Using slicing
def is_palindrome(s):
  return s == s[::-1]
print(is_palindrome("radar"))  # True
print(is_palindrome("hello"))  # False

# Using a loop
def is_palindrome_loop(s):
  for i in range(len(s)//2):
    if s[i] != s[-(i+1)]:
      return False
  return True
print(is_palindrome_loop("radar"))  # True
print(is_palindrome_loop("hello"))  # False
```

5. Write Python code to sort a list using the built-in sort() method.

```
# Example list
numbers = [5, 2, 9, 1, 5, 6]
# Sort the list in ascending order
numbers.sort()
print(numbers)  # [1, 2, 5, 5, 6, 9]
# Sort the list in descending order
```

```
numbers.sort(reverse=True)
print(numbers)  # [9, 6, 5, 5, 2, 1]
```

6. Write a Python function to replace all vowels in a string with spaces.

```
def replace_vowels(text):
   return ''.join(' ' if ch.lower() in 'aeiou' else ch for ch in text)

# Example usage
print(replace_vowels("Hello World"))  # H ll  W rld
print(replace_vowels("Python"))      # Pyth n
```

7. Write Python code to verify if a given string is a palindrome.

```
# Using slicing
def is_palindrome(s):
   return s == s[::-1]
print(is_palindrome("level"))  # True
print(is_palindrome("python")) # False

# Using a loop
def is_palindrome_loop(s):
   for i in range(len(s)//2):
      if s[i] != s[-(i+1)]:
         return False
   return True
print(is_palindrome_loop("level"))  # True
print(is_palindrome_loop("python")) # False
```

8. Count the number of occurrences of each word in a given string.

```
from collections import Counter

def count_words(text):
    words = text.lower().split()
    return Counter(words)


# Example usage
text = "Hello world hello Python world"
print(count_words(text))
# Output: Counter({'hello': 2, 'world': 2, 'python': 1})
```

9. Write Python code to sort a dictionary by its keys.

```
# Example dictionary
data = {'banana': 3, 'apple': 4, 'orange': 2, 'mango': 1}
# Sorting by keys
sorted_by_key = dict(sorted(data.items()))
print(sorted_by_key)
# Output: {'apple': 4, 'banana': 3, 'mango': 1, 'orange': 2}
# Sorting by keys in reverse order
sorted_by_key_desc = dict(sorted(data.items(), reverse=True))
print(sorted_by_key_desc)
# Output: {'orange': 2, 'mango': 1, 'banana': 3, 'apple': 4}
```

10. Find the first number in a list that appears consecutively three times.

```
def find_consecutive_triplet(nums):
    for i in range(len(nums) - 2):
        if nums[i] == nums[i+1] == nums[i+2]:
            return nums[i]
    return None
```

```
# Example usage
numbers = [1, 2, 2, 2, 3, 4]
print(find_consecutive_triplet(numbers))  # Output: 2

numbers = [1, 1, 2, 3, 3, 3]
print(find_consecutive_triplet(numbers))  # Output: 3
```

11. Find all pairs of numbers in a list that sum up to a given target number.

```
def find_pairs(nums, target):
    pairs = []
    seen = set()
    for num in nums:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
        seen.add(num)
    return pairs

# Example usage
numbers = [1, 2, 3, 4, 5, 6]
target = 7
print(find_pairs(numbers, target))
# Output: [(3, 4), (2, 5), (1, 6)]
```

12. *Write Python code to split a full name string into first name and last name*.

```
def split_name(full_name):
    first, last = full_name.strip().split(' ', 1)
```

```
    return first, last


# Example usage
name = "John Doe"
first_name, last_name = split_name(name)
print(first_name)  # John
print(last_name)   # Doe
name2 = "Alice Johnson"
f, l = split_name(name2)
print(f, l)       # Alice Johnson
```

13. Write Python code to find the character that appears most frequently in a string.

```
from collections import Counter
def max_repeated_char(s):
    s = s.replace(" ", "")  # optional: ignore spaces
    counts = Counter(s)
    return counts.most_common(1)[0]


# Example usage
text = "hello world"
print(max_repeated_char(text))  # ('l', 3)


text2 = "programming"
print(max_repeated_char(text2)) # ('g', 2)
```

14. Write Python code to find duplicate elements in a list and count how many times each occurs.

```
from collections import Counter
```

```
def find_duplicates(lst):
    counts = Counter(lst)
    return {k: v for k, v in counts.items() if v > 1}

# Example usage
numbers = [1, 2, 2, 3, 4, 4, 4, 5]
print(find_duplicates(numbers))
# Output: {2: 2, 4: 3}

names = ["Alice", "Bob", "Alice", "Eve"]
print(find_duplicates(names))
# Output: {'Alice': 2}
```

15. How can dictionaries be used in Python to efficiently store and access key-value pairs? Provide examples and tips.

```
# Basic dictionary
employee = {
    "name": "Alice",
    "department": "HR",
    "salary": 60000
}
# Accessing values
print(employee["name"])     # Output: Alice
print(employee.get("salary")) # Output: 60000
# Adding or updating
employee["salary"] = 65000
employee["location"] = "NY"
# Using defaultdict for counting
```

```
from collections import defaultdict, Counter


counter = defaultdict(int)
words = ["apple", "banana", "apple", "orange", "banana"]
for w in words:
    counter[w] += 1
print(counter)  # defaultdict(<class 'int'>, {'apple': 2, 'banana': 2, 'orange': 1})
# Using Counter for duplicates
word_count = Counter(words)
print(word_count)  # Counter({'apple': 2, 'banana': 2, 'orange': 1})
```

16. Write a Python program to calculate the factorial of a given number.

```
# Using recursion
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print(factorial(5))  # 120
print(factorial(0))  # 1


# Using iteration
def factorial_iter(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
print(factorial_iter(5))  # 120
```

17. Write Python code to load a CSV file into a pandas DataFrame and display the first 5 rows.

```
import pandas as pd

# Load CSV file into DataFrame

df = pd.read_csv("data.csv")

# Display the first 5 rows

print(df.head())
```

18. Write Python code to check whether a given number is an Armstrong number.

```
def is_armstrong(n):
   num_str = str(n)
   power = len(num_str)
   return n == sum(int(digit)**power for digit in num_str)


# Example usage

print(is_armstrong(153))  # True

print(is_armstrong(9474)) # True

print(is_armstrong(123))  # False
```

19. Write Python code to reshape a 1D NumPy array into a 2D array.

```
import numpy as np


# 1D array

arr = np.array([1, 2, 3, 4, 5, 6])


# Reshape into 2D (2 rows, 3 columns)

arr_2d = arr.reshape(2, 3)

print(arr_2d)
```

```
# Output:
# [[1 2 3]
#  [4 5 6]]

# Reshape into 3 rows, 2 columns
arr_2d_alt = arr.reshape(3, 2)
print(arr_2d_alt)
# Output:
# [[1 2]
#  [3 4]
#  [5 6]]
```

20. Write Python code to filter rows in a pandas DataFrame where a specific column value exceeds a given threshold (e.g., age > 30).

```
filtered_df = df[df["age"] > 30]
print(filtered_df)
```

21. Write Python code to get the ASCII value of a character and convert an ASCII value back to a character.

```
# Character to ASCII
char = 'A'
ascii_val = ord(char)
print(ascii_val)  # 65

# ASCII to character
ascii_val = 66
char = chr(ascii_val)
print(char)  # B
```

```
# Example with a string
text = "Hello"
ascii_values = [ord(c) for c in text]
print(ascii_values)  # [72, 101, 108, 108, 111]
```

22. Write Python code to find the indices of all non-zero elements in a NumPy array

```
import numpy as np
# Example array
arr = np.array([0, 2, 0, 5, 6, 0, 3])
# Get indices of non-zero elements
non_zero_indices = np.nonzero(arr)
print(non_zero_indices)      # (array([1, 3, 4, 6]),)
# If you want a flat list
print(non_zero_indices[0].tolist())  # [1, 3, 4, 6]
```

23. Write Python code to add a new column to a pandas DataFrame that is calculated from existing columns (e.g., total = price × quantity).

```
df["total"] = df["price"] * df["quantity"]
print(df)
```

24. Write Python code to calculate the sum of squares of the first n natural numbers.

```
# Using a loop
def sum_of_squares(n):
   return sum(i**2 for i in range(1, n+1))
print(sum_of_squares(5))  # 55 (1^2 + 2^2 + 3^2 + 4^2 + 5^2)
# Using formula: n(n+1)(2n+1)/6
```

```
def sum_of_squares_formula(n):
   return n*(n+1)*(2*n+1)//6
print(sum_of_squares_formula(5))  # 55
```

25. Write Python code to solve a system of linear equations using NumPy.

```
import numpy as np

# Example system:
# 2x + 3y = 8
# 5x + 7y = 19
# Coefficient matrix
A = np.array([[2, 3],
          [5, 7]])
# Constants vector
B = np.array([8, 19])
# Solve for x and y
solution = np.linalg.solve(A, B)
print(solution)  # [1. 2.] -> x=1, y=2
```

26. Write Python code to drop all rows in a pandas DataFrame that have missing (NaN) values in any column.

```
df_clean = df.dropna()
print(df_clean)
```

27. Write Python code to find all common letters between two strings.

```
# Using set intersection
def common_letters(str1, str2):
   return set(str1) & set(str2)
```

```
# Example usage
s1 = "python"
s2 = "typhoon"
print(common_letters(s1, s2))  # {'p', 't', 'h', 'o', 'n'}
s3 = "hello"
s4 = "world"
print(common_letters(s3, s4))  # {'o', 'l'}
```

28. Write Python code to group a pandas DataFrame by a categorical column and calculate the average of a numerical column.

```
import pandas as pd
# Sample DataFrame
data = {
   "department": ["HR", "IT", "HR", "IT", "Finance"],
   "salary": [50000, 60000, 55000, 65000, 70000]
}
df = pd.DataFrame(data)
# Group by 'department' and calculate mean salary
mean_salary = df.groupby("department")["salary"].mean()
print(mean_salary)
```

29. Write Python code to create a dictionary by combining two lists: one for keys and one for values.

```
# Example lists
keys = ["name", "age", "city"]
values = ["Alice", 25, "New York"]
```

```
# Convert to dictionary
my_dict = dict(zip(keys, values))
print(my_dict)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

30. Write Python code to perform element-wise division of two NumPy arrays

```
import numpy as np

# Example arrays
arr1 = np.array([10, 20, 30, 40])
arr2 = np.array([2, 4, 5, 8])
# Element-wise division
result = arr1 / arr2
print(result)
# Output: [5. 5. 6. 5.]
# Handling integer division if needed
result_int = arr1 // arr2
print(result_int)
# Output: [5 5 6 5]
```

31. Write Python code to transform a pandas DataFrame from wide format to long format using melt().

```
import pandas as pd
# Sample wide DataFrame
data = {
   "id": [1, 2, 3],
   "math": [90, 80, 85],
   "science": [95, 85, 88]
```

```
}
df = pd.DataFrame(data)
# Melt DataFrame from wide to long format
df_long = pd.melt(df, id_vars=["id"], var_name="subject", value_name="score")
print(df_long)
```

32. Write Python code to reverse a given string.

```
# Using slicing
def reverse_string(s):
   return s[::-1]
print(reverse_string("hello"))  # olleh
print(reverse_string("Python")) # nohtyP

# Using reversed() and join()
def reverse_string_alt(s):
   return ''.join(reversed(s))
print(reverse_string_alt("hello"))  # olleh
```

33. Write Python code to remove duplicate rows from a pandas DataFrame while keeping the first occurrence.

```
import pandas as pd

# Sample DataFrame with duplicates
data = {
   "name": ["Alice", "Bob", "Alice", "David"],
   "age": [25, 30, 25, 40]
}
df = pd.DataFrame(data)
```

```
# Drop duplicate rows, keep first occurrence
df_unique = df.drop_duplicates(keep='first')
print(df_unique)
```

34. Write Python code to count the number of words in a paragraph.

```
# Using split()
def word_count(paragraph):
    words = paragraph.split()
    return len(words)

# Example usage
text = "Python is a powerful programming language."
print(word_count(text))  # Output: 6
# Using collections.Counter to count frequency of each word
from collections import Counter

def word_frequency(paragraph):
    words = paragraph.lower().split()
    return Counter(words)
print(word_frequency(text))
# Output: Counter({'python': 1, 'is': 1, 'a': 1, 'powerful': 1, 'programming': 1, 'language.': 1})
```

35. Write Python code to replace values in a pandas DataFrame column using a dictionary mapping with the map() function.

```
import pandas as pd
# Sample DataFrame
```

```
data = {
   "fruit": ["apple", "banana", "cherry", "banana", "apple"]
}
df = pd.DataFrame(data)
# Dictionary mapping
replace_dict = {"apple": "A", "banana": "B", "cherry": "C"}

# Replace values using map()
df["fruit_code"] = df["fruit"].map(replace_dict)
print(df)
```

36. Write Python code to find the minimum and maximum values in a NumPy array.

```
import numpy as np

# Example array
arr = np.array([5, 2, 9, 1, 7, 6])

# Find smallest and largest numbers
smallest = np.min(arr)
largest = np.max(arr)

print("Smallest:", smallest)  # Smallest: 1
print("Largest:", largest)    # Largest: 9

# Alternative using Python built-ins
print("Smallest:", min(arr))
print("Largest:", max(arr))
```

37. Write Python code to merge two strings by alternating their characters.

```
def merge_alternate(s1, s2):
    merged = ''.join(a + b for a, b in zip(s1, s2))
    # Append the remaining characters if strings are of unequal length
    merged += s1[len(s2):] + s2[len(s1):]
    return merged

# Example usage
s1 = "abc"
s2 = "123"
print(merge_alternate(s1, s2))  # a1b2c3
s3 = "hello"
s4 = "world!"
print(merge_alternate(s3, s4))  # hweolrllod!
```

38. Write Python code to filter rows in a pandas DataFrame where a timestamp column falls between two specific dates.

```
import pandas as pd

# Sample DataFrame
data = {
    "timestamp": ["2026-01-10", "2026-01-15", "2026-01-20", "2026-01-25"],
    "value": [10, 20, 30, 40]
}
df = pd.DataFrame(data)

# Convert 'timestamp' column to datetime
df["timestamp"] = pd.to_datetime(df["timestamp"])
```

```
# Define date range
start_date = "2026-01-12"
end_date = "2026-01-22"

# Filter rows between the two dates
filtered_df = df[(df["timestamp"] >= start_date) & (df["timestamp"] <= end_date)]
print(filtered_df)
```

39. Write Python code to find the greatest common divisor (GCD) of two strings, i.e., the largest string that can be concatenated multiple times to form both strings.

```
import math
def gcd_of_strings(str1, str2):
  if str1 + str2 != str2 + str1:
    return ""
  gcd_length = math.gcd(len(str1), len(str2))
  return str1[:gcd_length]

# Example usage
print(gcd_of_strings("ABABAB", "ABAB"))  # AB
print(gcd_of_strings("ABCABC", "ABC"))   # ABC
print(gcd_of_strings("LEET", "CODE"))    # ""
```

40. Write Python code to reverse only the vowels in a given string.

```
def reverse_vowels(s):
  vowels = "aeiouAEIOU"
  s = list(s)
  i, j = 0, len(s) - 1
```

```
  while i < j:
    if s[i] not in vowels:
      i += 1
    elif s[j] not in vowels:
      j -= 1
    else:
      s[i], s[j] = s[j], s[i]
      i += 1
      j -= 1
  return ''.join(s)

# Example usage
print(reverse_vowels("hello"))      # holle
print(reverse_vowels("leetcode"))   # leotcede
```

41. Write Python code to merge multiple pandas DataFrames and calculate aggregations (e.g., sum or mean) on a column.

```
import pandas as pd

# Sample DataFrames
df1 = pd.DataFrame({
  "id": [1, 2, 3],
  "sales": [100, 200, 300]
})

df2 = pd.DataFrame({
  "id": [2, 3, 4],
  "sales": [150, 250, 400]
```

```
})

df3 = pd.DataFrame({
   "id": [1, 4, 5],
   "sales": [120, 300, 500]
})

# Merge DataFrames (outer join on 'id')
merged_df = pd.concat([df1, df2, df3])

# Aggregate: sum of sales per id
agg_df = merged_df.groupby("id")["sales"].sum().reset_index()
print(agg_df)
```

42. Write Python code to move all zeros in a list to the end while maintaining the order of non-zero elements.

```
def move_zeros_to_end(lst):
   result = [x for x in lst if x != 0]  # Keep non-zero elements
   zeros = [0] * lst.count(0)         # Count zeros
   return result + zeros

# Example usage
numbers = [0, 1, 0, 3, 12, 0, 5]
print(move_zeros_to_end(numbers))
# Output: [1, 3, 12, 5, 0, 0, 0]
```

43. Write Python code to check if one string is a subsequence of another string.

```
def is_subsequence(s1, s2):
```

```
    # Check if s1 is a subsequence of s2
    it = iter(s2)
    return all(c in it for c in s1)

# Example usage
print(is_subsequence("abc", "aebdc"))  # True
print(is_subsequence("axc", "ahbgdc")) # False
print(is_subsequence("ace", "abcde"))  # True
```

44. Write Python code to flatten a nested JSON object into a flat dictionary or DataFrame.

```
import pandas as pd
from pandas import json_normalize

# Sample nested JSON
data = [
  {
    "id": 1,
    "name": "Alice",
    "address": {"city": "New York", "zip": "10001"}
  },
  {
    "id": 2,
    "name": "Bob",
    "address": {"city": "Los Angeles", "zip": "90001"}
  }
]
```

```
# Flatten JSON to DataFrame
df = json_normalize(data)
print(df)

# Output:
#    id   name address.city address.zip
# 0   1  Alice     New York      10001
# 1   2    Bob  Los Angeles      90001
```

45. Write Python code to find the unique elements in a list and count how many times each element occurs.

```
from collections import Counter

# Sample list
numbers = [1, 2, 2, 3, 3, 3, 4]

# Count occurrences
occurrences = Counter(numbers)
print(occurrences)
# Output: Counter({3: 3, 2: 2, 1: 1, 4: 1})

# If you want unique elements only
unique_numbers = list(occurrences.keys())
print(unique_numbers)
# Output: [1, 2, 3, 4]
```